

Laboratorio 4: Trabajo integrador

Objetivo general

El presente trabajo tiene como finalidad integrar los contenidos teóricos y prácticos del curso mediante la implementación completa de un proyecto con enfoque DevOps y DevSecOps.

Se debe tomar un proyecto existente y mejorar su arquitectura, seguridad, despliegue y monitoreo aplicando las herramientas vistas en el curso:

Docker, Kubernetes, Helm, Jenkins, Grafana, Prometheus, Falco, Snyk, Semgrep y Kyverno.

Objetivos específicos

El proyecto debe ejecutarse de forma contenedorizada, ser desplegable en Kubernetes mediante Helm, incluir un pipeline de integración continua y entrega continua (CI/CD) configurado en Jenkins, contar con monitoreo activo de métricas y seguridad, e implementar controles automatizados de análisis estático, escaneo de vulnerabilidades y políticas de cumplimiento.

Desglose de requisitos

1. Contenerización (Docker)

- a. Se debe contenerizar cada componente del sistema (backend, frontend, servicios auxiliares, etc.) utilizando Docker.
- b. Se debe crear un Dockerfile por componente.
- c. Las imágenes deben:
 - a. Utilizar versiones específicas (no latest).
 - b. No ejecutar procesos como usuario root.
 - c. Ser optimizadas en tamaño utilizando multi-stage builds.
- d. Se debe validar la calidad de la imagen con:
 - a. Snyk o Trivy para detectar vulnerabilidades.
 - b. Dive (en Windows) o SlimToolkit (en macOS/Linux) para analizar el tamaño y la composición de las capas.

- e. Se debe incluir un breve reporte (/reports/image-analysis.md) que describa:
 - a. El tamaño total de la imagen.
 - b. La cantidad de capas.
 - c. Las observaciones de optimización posibles.

2. Orquestación y despliegue (Kubernetes + Helm)

- a. Se debe crear una Helm Chart que permita el despliegue del proyecto en un cluster de Kubernetes.
- b. La Chart debe contener, como mínimo:
 - a. Plantillas para Deployment, Service, ConfigMap y Ingress (si corresponde).
 - b. Un archivo values.yaml con parámetros configurables (puerto, imagen, réplicas, recursos, etc.).
- c. Se debe ejecutar el despliegue utilizando el comando:
 - a. `helm install <nombre> ./helm-chart`
- d. Se debe incluir un archivo values-prod.yaml o values-dev.yaml con parámetros diferenciados por entorno.
- e. Se debe verificar que los pods inicien correctamente y que el servicio sea accesible.

3. Integración y entrega continua (Jenkins)

- a. Se debe crear un pipeline en Jenkins que automatice el flujo de integración y despliegue.
- b. El pipeline debe incluir las siguientes etapas:
 - a. Clonación del repositorio.
 - b. Análisis de código estático utilizando Semgrep.
 - c. Escaneo de vulnerabilidades de dependencias utilizando Snyk.
 - d. Construcción y test de la aplicación.
 - e. Construcción y publicación de la imagen Docker.
 - f. Despliegue automático en Kubernetes utilizando Helm.
- c. Se debe incluir un archivo Jenkinsfile en el repositorio con las etapas configuradas.
- d. El pipeline debe ejecutarse correctamente y producir registros de cada fase.
- e. En caso de fallas por vulnerabilidades críticas, el pipeline debe detener la ejecución y reportar el error.

4. Monitoreo y observabilidad (Prometheus + Grafana)

- a. Se debe instrumentar la aplicación para exponer métricas accesibles por Prometheus.
- b. Ejemplo: endpoint /metrics o integración mediante Micrometer o librerías equivalentes.
- c. Se debe desplegar Prometheus en el cluster y configurar el scrape del servicio.
- d. Se debe desplegar Grafana y conectarlo como fuente de datos a Prometheus.
- e. Se debe crear un dashboard personalizado en Grafana que muestre, como mínimo:
 - a. Número de peticiones procesadas (RPS).
 - b. Latencia promedio de respuesta.
 - c. Uso de CPU y memoria por pod.
 - d. Una métrica de negocio relevante para la aplicación.
- f. Se debe exportar el dashboard en formato .json y guardar el archivo en /grafana/dashboard.json.

5. Seguridad integrada (DevSecOps)

5.1 Análisis estático de código (Semgrep)

- a. Se debe ejecutar Semgrep sobre el código fuente del proyecto.
- b. Se deben aplicar al menos dos reglas de seguridad específicas al lenguaje utilizado.
- c. Se debe guardar el resultado en /reports/semgrep-report.txt.
- d. Las vulnerabilidades detectadas deben corregirse o justificarse brevemente.

5.2 Escaneo de dependencias (Snyk)

- a. Se debe ejecutar Snyk sobre las dependencias del proyecto.
- b. Se debe guardar el resultado en /reports/snyk-report.txt.
- c. Se deben identificar las vulnerabilidades críticas y proponer correcciones.

5.3 Políticas de seguridad en Kubernetes (Kyverno)

- a. Se debe instalar Kyverno en el cluster.
- b. Se deben crear al menos cuatro políticas:

- a. Una política que prohíba el uso de imágenes con la etiqueta latest.
 - b. Una política que exija límites de CPU y memoria en todos los pods.
 - c. Una política que impida la ejecución de contenedores como root.
 - d. Una política adicional que forme parte de las buenas prácticas que considere necesaria.
- c. Se debe aplicar cada política y verificar que los pods que incumplan sean rechazados.
- d. Se debe registrar la evidencia de las validaciones en `/reports/kyverno-validation.log`.

5.4 Monitoreo de seguridad en tiempo de ejecución (Falco)

- a. Se debe instalar Falco en el cluster (manual o mediante Helm Chart oficial).
- b. Se debe ejecutar al menos una acción que dispare una alerta (esta acción no puede ser abrir un shell en un contenedor).
- c. Se debe verificar que Falco registre el evento y guardar la alerta en `/reports/falco-event.log`.
- d. Se debe incluir una breve descripción del evento detectado.

6. Reporte técnico final

- a. Se debe generar un archivo README.md o informe PDF con los siguientes apartados:
 - a. Descripción general del proyecto.
 - b. Arquitectura final (con diagrama de componentes).
 - c. Pipeline implementado (etapas y herramientas).
 - d. Métricas y dashboard de Grafana.
 - e. Resultados de análisis de seguridad (Semgrep, Snyk, Kyverno, Falco).
 - f. Problemas encontrados y soluciones adoptadas.
 - g. Conclusiones generales y oportunidades de mejora.
- b. Se deben incluir capturas o fragmentos de salida de las herramientas principales.
- c. Se debe adjuntar el dashboard de Grafana exportado.

Consideraciones adicionales

Para este entregable, se debe:

- a) Tener scripts de bash que se encarguen de inicializar, hacer solicitudes a la aplicación de manera automática para popular los dashboards, y borrar todos los recursos. También se debe incluir en estos comandos la instalación de operadores de Kubernetes.
- b) Tener scripts que inicien todos los componentes necesarios (Jenkins, Grafana, etc.)
- c) Tener documentación extensa que explique paso a paso qué comandos utilizar y los resultados que se obtuvieron en las etapas donde haya que visualizar resultados.